

Structure et organisation d'un projet Java

1. Pourquoi structurer un projet Java ?

Une bonne structure permet de :

- mieux lire le code
- séparer les responsabilités
- faciliter la maintenance
- rendre le projet plus professionnel
- travailler plus facilement en équipe

Un projet Java mal organisé devient vite difficile à faire évoluer.

2. Structure standard d'un projet Java

Dans un projet moderne, on utilise souvent **Maven** ou **Gradle**.

La structure la plus standard avec Maven est :

```
mon-projet/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/entreprise/monprojet/
│   │   │   │   ├── Main.java
│   │   │   │   ├── model/
│   │   │   │   ├── service/
│   │   │   │   ├── repository/
│   │   │   │   ├── controller/
│   │   │   │   └── utils/
│   │   └── resources/
│   │       ├── application.properties
│   │       └── data.txt
│   └── test/
│       ├── java/
│       │   ├── com/entreprise/monprojet/
│       │   └── MainTest.java
├── pom.xml
└── README.md
```

3. Rôle des dossiers

src/main/java

Contient le **code source principal** de l'application.

src/main/resources

Contient les **fichiers de configuration** et autres ressources :

- fichiers .properties
- fichiers .xml
- fichiers JSON
- templates
- fichiers texte

src/test/java

Contient les **tests unitaires**.

pom.xml

Fichier de configuration Maven :

- dépendances
- version Java
- plugins
- build

4. Organisation par packages

En Java, on organise le code dans des **packages**.

Exemple :

`com.entreprise.monprojet`

Puis on découpe par responsabilité :

model

Contient les objets métier.

Exemple :

```
public class Etudiant {  
    private String nom;  
    private int age;  
}
```

service

Contient la logique métier.

Exemple :

```
public class EtudiantService {  
    public void ajouterEtudiant(Etudiant e) {  
        // logique métier  
    }  
}
```

repository

Contient l'accès aux données :

- base de données
- fichiers
- API externes

Exemple :

```
public class EtudiantRepository {  
    public void save(Etudiant e) {  
        // sauvegarde  
    }  
}
```

controller

Contient le point d'entrée de l'application.

Selon le type de projet :

- application console
- API REST
- application web

Exemple :

```
public class EtudiantController {  
    // reçoit les requêtes et appelle le service  
}
```

utils

Fonctions utilitaires réutilisables :

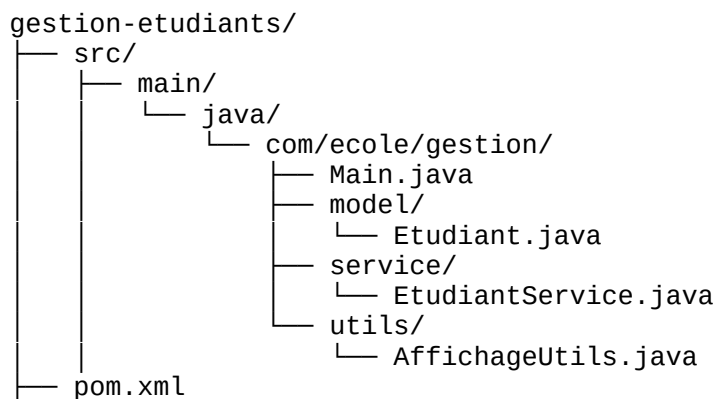
- validation
- formatage
- conversions
- helpers

Exemple :

```
public class DateUtils {  
    public static String formatDate(LocalDate date) {  
        return date.toString();  
    }  
}
```

5. Exemple simple : projet Java console

Arborescence



Rôle des classes

- `Main.java` : démarre le programme
- `Etudiant.java` : représente l'étudiant
- `EtudiantService.java` : gère les traitements
- `AffichageUtils.java` : centralise les affichages

6. Exemple de structure pour une API REST Java

Si tu développes avec **Spring Boot**, la structure classique est :

```
mon-api/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/entreprise/api/
│   │   │       ├── ApiApplication.java
│   │   │       ├── controller/
│   │   │       ├── service/
│   │   │       ├── repository/
│   │   │       ├── entity/
│   │   │       ├── dto/
│   │   │       ├── mapper/
│   │   │       ├── config/
│   │   │       └── exception/
│   │   └── resources/
│   │       ├── application.properties
│   │       └── schema.sql
│   └── test/
└── pom.xml
```

7. Packages souvent utilisés dans un projet professionnel

entity ou model

Objets liés au métier ou à la base de données.

dto

Objets d'échange entre couches ou avec l'extérieur.

Exemple :

```
public class EtudiantDTO {
    private String nom;
    private int age;
}
```

mapper

Transformation entre Entity et DTO.

config

Configuration technique :

- sécurité

- CORS
- beans
- base de données

exception

Gestion centralisée des erreurs.

service

Logique métier.

repository

Accès aux données.

controller

Endpoints REST.

8. Bonne organisation en couches

Une organisation propre suit souvent ce schéma :

Controller -> Service -> Repository -> Database

Explication

- **Controller** : reçoit la requête
- **Service** : applique la logique métier
- **Repository** : dialogue avec la base
- **Database** : stocke les données

C'est une architecture simple, claire et très utilisée.

9. Exemple de flux complet

Cas : ajouter un étudiant

1. le Controller reçoit la demande
2. le Service vérifie les règles métier
3. le Repository enregistre
4. la base de données stocke

10. Exemple de nommage des packages

On évite :

```
test1/  
classe1/  
mestrucs/  
divers/
```

On préfère :

```
com.ecole.gestion.etudiant  
com.entreprise.facturation.service  
com.startup.marketplace.repository
```

Le nom doit être :

- clair
- cohérent
- stable

11. Règles importantes d'organisation

Une classe = une responsabilité

Éviter les classes géantes qui font tout.

Séparer métier et technique

La logique métier ne doit pas être mélangée avec :

- affichage
- accès base
- configuration

Éviter le code dupliqué

Créer des services ou utilitaires réutilisables.

Ranger les fichiers par rôle

Pas par hasard.

Utiliser des noms explicites

Exemple :

- UtilisateurService
- CommandeRepository
- FactureController

Pas :

- Service1
- Utils2

12. Exemple d'organisation orientée métier

Pour un projet plus grand, on peut organiser par **module métier** :

```
com.entreprise.application
├── utilisateur/
│   ├── controller/
│   ├── service/
│   ├── repository/
│   └── model/
├── facture/
│   ├── controller/
│   ├── service/
│   ├── repository/
│   └── model/
└── produit/
    ├── controller/
    ├── service/
    ├── repository/
    └── model/
```

Cette approche est très intéressante sur les projets métier riches.

13. Exemple d'organisation pour débutant

Pour un petit projet pédagogique, tu peux commencer simplement :

```
src/main/java/com/ecole/projet/
├── Main.java
├── model/
├── service/
└── utils/
```

Puis, quand le projet grandit, ajouter :

- repository
- controller
- dto
- exception

14. Exemple minimal de classes

Main.java

```
package com.ecole.projet;  
import com.ecole.projet.model.Etudiant;  
import com.ecole.projet.service.EtudiantService;  
public class Main {  
    public static void main(String[] args) {  
        EtudiantService service = new EtudiantService();  
        service.ajouterEtudiant(new Etudiant("Jean", 20));  
    }  
}
```

Etudiant.java

```
package com.ecole.projet.model;  
public class Etudiant {  
    private String nom;  
    private int age;  
    public Etudiant(String nom, int age) {  
        this.nom = nom;  
        this.age = age;  
    }  
    public String getNom() {  
        return nom;  
    }  
    public int getAge() {  
        return age;  
    }  
}
```

EtudiantService.java

```
package com.ecole.projet.service;  
import com.ecole.projet.model.Etudiant;  
public class EtudiantService {  
    public void ajouterEtudiant(Etudiant e) {  
        System.out.println("Etudiant ajouté : " + e.getNom());  
    }  
}
```

15. Outils de build

Maven

Le plus classique en Java entreprise.

Fichier :

pom.xml

Gradle

Plus flexible, souvent utilisé aussi.

Fichiers :

build.gradle

settings.gradle

16. Organisation recommandée pour tes étudiants

Pour une formation d'initiation, je te recommande cet ordre :

Niveau 1

- Main
- variables
- méthodes
- classes simples

Niveau 2

- model
- service
- utils

Niveau 3

- repository
- gestion des fichiers ou base
- exceptions

Niveau 4

- architecture en couches
- API REST avec Spring Boot

17. Résumé

Structure standard

```
src/main/java  
src/main/resources  
src/test/java
```

Packages classiques

- model
- service

- repository
- controller
- utils
- config
- exception
- dto

Principe clé

Un projet Java bien organisé sépare le métier, la logique, l'accès aux données et la configuration.

18. Structure recommandée simple

Pour commencer proprement :

```
mon-projet-java/  
├── src/  
│   ├── main/  
│   │   ├── java/com/monapp/  
│   │   │   ├── Main.java  
│   │   │   ├── model/  
│   │   │   ├── service/  
│   │   │   ├── repository/  
│   │   │   └── utils/  
│   │   └── resources/  
│   └── test/  
├── pom.xml  
└── README.md
```